Metadata of the chapter that will be visualized in SpringerLink

Book Title	Theoretical Aspects of Computing	
Series Title		
Chapter Title	SAT-Based Consistency Checking for Structural and Behavioral SysML V2 Models	
Copyright Year	2027 lowercase v	
Copyright HolderName	The Author(s), under exclusive license to Springer Nature Switzerland AG	
Corresponding Author	Family Name	**Aguirre**
	Particle	
	Given Name	**Nazareno**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	University of Rio Cuarto / CONICET
	Address	Cárdoba, Argentina Río Cuarto
	Division	
	Organization	Guangdong Technion Israel Institute of Technology
	Address	Shantou, China
	Division	
	Organization	~~Universidad de San Andrés~~
	Address	~~Buenos Aires, Argentina~~
	Email	naguirre@dc.exa.unrc.edu.ar
Author	Family Name	**Garbervetsky**
	Particle	
	Given Name	**Diego**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	University of Buenos Aires / CONICET
	Address	~~Cárdoba, Argentina~~ Buenos Aires, Argentina
	Division	
	Organization	Guangdong Technion Israel Institute of Technology
	Address	Shantou, China
	Division	
	Organization	~~Universidad de San Andrés~~
	Address	~~Buenos Aires, Argentina~~
	Email	
Author	Family Name	**Uchitel**
	Particle	
	Given Name	**Sebastián**

	Prefix	
	Suffix	
	Role	
	Division	
	Organization	University of Buenos Aires / CONICET
	Address	~~Córdoba, Argentina~~ Buenos Aires, Argentina
	Division	
	Organization	~~Guangdong Technion Israel Institute of Technology~~ Universidad de San Andrés
	Address	~~Shantou, China~~ Buenos Aires, Argentina
	Email	
Author	Family Name	**Yue**
	Particle	
	Given Name	**Tao**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	Beihang University
	Address	Beijing, China
	Email	
Author	Family Name	**Zhang**
	Particle	
	Given Name	**Man**
	Prefix	
	Suffix	
	Role	
	Division	
	Organization	Beihang University
	Address	Beijing, China
	Email	

Abstract	The second generation of the Systems Modeling Language (SysML v2) introduces a unified textual notation that integrates structural definitions, state-machine behaviors, and temporal constraints, enabling precise formulations of system properties. However, while the translation of SysML v1 models into logical languages for satisfiability checking (e.g., Alloy) has been extensively studied, the new language features in SysML v2, particularly its richer behavioral constructs, including native state-machine queries and temporal occurrence ordering, pose new challenges for existing transformation strategies when checking structural and temporal consistency. In this paper, we argue that these behavioral aspects are not well captured by standard Alloy encodings, and that dynamic extensions of Alloy, such as Alloy 6 and DynAlloy, offer a more natural translation and a more effective basis for analysis. These languages already provide efficient and expressive encodings for stateful behaviors and action sequencing, which can be directly leveraged to handle SysML v2's behavioral semantics. We identify the underlying semantic gaps in standard Alloy for representative SysML v2 constraints and discuss why dynamic Alloy variants are better suited to address them. We illustrate our observations on a representative SysML v2 model.

Author Proof

Check for updates

SAT-Based Consistency Checking for Structural and Behavioral SysML V2 Models

Nazareno Aguirre[1,4,5(✉)], Diego Garbervetsky[2,4,5], Sebastián Uchitel[2,4], Tao Yue[3], and Man Zhang[3]

[1] University of Rio Cuarto / CONICET, Córdoba, Argentina
naguirre@dc.exa.unrc.edu.ar
[2] University of Buenos Aires / CONICET, Córdoba, Argentina
[3] Beihang University, Beijing, China
[4] Guangdong Technion Israel Institute of Technology, Shantou, China
[5] Universidad de San Andrés, Buenos Aires, Argentina

Abstract. The second generation of the Systems Modeling Language (SysML v2) introduces a unified textual notation that integrates structural definitions, state-machine behaviors, and temporal constraints, enabling precise formulations of system properties. However, while the translation of SysML v1 models into logical languages for satisfiability checking (e.g., Alloy) has been extensively studied, the new language features in SysML v2, particularly its richer behavioral constructs, including native state-machine queries and temporal occurrence ordering, pose new challenges for existing transformation strategies when checking structural and temporal consistency. In this paper, we argue that these behavioral aspects are not well captured by standard Alloy encodings, and that dynamic extensions of Alloy, such as Alloy 6 and DynAlloy, offer a more natural translation and a more effective basis for analysis. These languages already provide efficient and expressive encodings for stateful behaviors and action sequencing, which can be directly leveraged to handle SysML v2's behavioral semantics. We identify the underlying semantic gaps in standard Alloy for representative SysML v2 constraints and discuss why dynamic Alloy variants are better suited to address them. We illustrate our observations on a representative SysML v2 model.

1 Introduction

The Systems Modeling Language version 2 (SysML v2) [19] replaces the UML-based profile architecture of SysML v1 with a textual language built on the Kernel Modeling Language (KerML) [18]. Unlike its predecessor, where structure and behavior were specified through separate stereotypes and diagrams with their own semantics [20], SysML v2 unifies structure, behavior, and constraints under a single semantic framework. Parts and their features, state machines with `accept/then` transitions, and occurrence- and time-ordered behaviors are all first-class constructs within the same textual notation. Constraints are expressed

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
P. R. D'Argenio and A. Sokolova (Eds.): ICTAC 2026, LNCS 17239, pp. 1–11, 2027.
https://doi.org/10.1007/978-3-032-43868-3_8

Author Proof

directly in the SysML v2 expression language, which navigates the structural model and can be conditioned on the active state of a state machine, rather than through a separate OCL layer. This integration enables more coherent specifications of system properties, particularly those that involve both structure and behavior.

A standard analysis on system models is consistency checking: determining whether the structural and behavioral constraints of a system are jointly satisfiable. Beyond guaranteeing the absence of contradictions, consistency checking also serves a crucial role in validation: the analysis can produce witness instances that help developers identify unintended or surprising models, exposing subtle issues early in the design process [22,25]. This task is often non-obvious; subtle interactions between inheritance, multiplicities, and behavioral specifications can render a model inconsistent, or admit unintended models, even when each fragment appears consistent in isolation. For SysML v1, consistency checking has largely been approached via satisfiability checking, with translations into SAT-based languages being the prevalent technique [3]. Alloy [12,13] has proven particularly effective for this purpose, due to its relational nature, its state-based semantics, and the significant body of work devoted to efficient SAT-based instance generation—including support for partial instance specifications via relational bounds [23] and symmetry breaking predicates that reduce the number of variables in the SAT encoding [14]. These features are especially relevant for analyzing SysML models, where large structural models benefit from such optimizations. The typical target for such analyses, however, has been structural constraints, with behavioral aspects often relegated to simple invariants.

SysML v2 places a stronger emphasis on behavioral models. This shift introduces new challenges for consistency analysis. Although there is no single standard notion of behavioral consistency, typical properties—such as whether every state is reachable along some trace while satisfying the structural constraints at each step—are natural candidates. These properties go beyond what can be captured by structural well-formedness alone. Such behavioral notions—reachability, safety over executions, and trace-level properties—require reasoning that goes beyond a single configuration. While standard Alloy is in principle expressive enough to encode these properties [15], the encoding is often cumbersome and impractical. Dynamic extensions of Alloy, such as Alloy 6 (Electrum) [4] and DynAlloy [10,21], are better suited for this task. These languages are explicitly designed to capture stateful behaviors and action sequencing, providing more natural encodings that also benefit from better analysis performance. We argue that these extensions offer a more practical foundation for analyzing SysML v2's behavioral consistency than a direct adaptation of the traditional SysML v1-to-Alloy approach.

Given these challenges, one might consider using dedicated state-machine analyzers such as LTSA or NuSMV, which efficiently decide reachability and temporal properties. However, these tools focus primarily on control, offering little support for the rich structural features of SysML v2— e.g., multiplicities, quantified navigation over collections—that need to be preserved alongside behavioral

Author Proof

Listing 1: Fragment of a SysML v2 model of a fire alarm.

```
package FireAlarm {
 item def Tick; // event: a time step elapses
 part def Sensor {
  attribute isDetecting : Boolean;
 }
 part def Controller {
  part sensors : Sensor[1..*];
  attribute isOn : Boolean = false;
 // --- State Machine ---
 state def ControllerStates {
  entry; then idle;
  state idle {
  // coupling: the controller reads its
      sensors' 'isDetecting' attribute
   accept Tick
    if sensors->exists {in s : Sensor | s.
     isDetecting} then active;
  }
  state active {
    entry assign isOn := true;
    accept Tick then alarm;
   }
   state alarm {
    accept Tick
     if sensors->forall {in s : Sensor; not s
      .isDetecting} then disabled;
    accept Tick
     if sensors->exists {in s : Sensor; s.
      isDetecting} then alarm;
   }
   state disabled {
    accept Tick
     if sensors->exists {in s : Sensor; s.
      isDetecting} then alarm;
    accept Tick
     if sensors->forall {in s : Sensor; not s
      .isDetecting} then disabled;
   }
  }
  state controller : ControllerStates;
  // --- End State Machine ---
 }
 part def FireAlarmSystem {
  // System owns the physical sensors
  part physicalSensors : Sensor[3];
  // System owns a controller for  the
      sensors
  part ctrl : Controller;
  // Controller is only armed if at least one
       sensor is detecting
  constraint NoFalseAlarm {
   ctrl.isOn implies ctrl.sensors->exists {in
       s : Sensor; s.isDetecting}
  }
 }
}
```

reasoning. We therefore remain within Alloy, where structure and behavior share a common representation. For simple state-machine properties such as reachability and safety over executions, Alloy 6 (Electrum) provides a natural encoding via its temporal operators. When the behavioral model involves complex control flow, however, DynAlloy offers a better fit, supporting action composition and contract-based reasoning while keeping constraints over navigated structures exact. We argue that this combination—standard Alloy for structure, Alloy 6 for trace-level properties, and DynAlloy for action-oriented fragments—provides a practical foundation for analyzing consistency in SysML v2 models but requires developing a common semantic framework for Alloy 6 and DynAlloy.

This paper contributes an analysis of why standard Alloy encodings are insufficient for behavioral consistency properties in SysML v2 together with an initial proposal of a hybrid approach that allocates structural and behavioral concerns across Alloy, Alloy 6, and DynAlloy. To the best of our knowledge, no prior work targets SysML v2 with Alloy or DynAlloy; formal analysis of SysML v2 is still in its early stages, whereas the SysML v1-to-Alloy line is well-established but does not address behavioral aspects [24].

2 Background

SysML v2 and KerML. SysML v2 models are written in KerML, a kernel with formal semantics that unifies structure and behavior. A `part def` is a classifier that may specialize several general classifiers; its features include attributes,

Author Proof

ports, parts, and state machines. Features may declare `subsets` or `redefines` relations, which KerML resolves during model elaboration into a single effective feature set. State machines are declared with `state def`; states are introduced with `state`, the initial transition with `entry; then` s, and event-triggered transitions with `accept` E `then` s. Behavior is grounded in an occurrence/time model in which *successions* (*HappensBefore* connectors) order occurrences in time; the constraint and requirement language itself, however, remains a Boolean state-predicate language. Listing 1 shows a fragment of a fire-alarm model, in valid SysML v2 textual notation, used throughout the paper. As written it contains a latent structural defect that we deliberately leave in place to illustrate the use of consistency checking in modeling in general, and in SysML in particular.

Alloy, Electrum, and DynAlloy. Alloy [12] models are built from `sig` (sets of atoms) and `fact` (enforced invariant) declarations, and the Alloy Analyzer performs bounded model finding and counterexample generation, resorting to SAT solving. Classical Alloy is static. Alloy 6 integrates the temporal logic of Electrum [4], adding mutable signatures/fields and LTL operators (`always`, `eventually`, `after`, `until`) over (possibly infinite, lasso-shaped) execution traces, which makes it the natural target for expressing reachability, safety, and liveness over state machines. DynAlloy [10] takes a complementary, operational route: an *action* is a pair of pre- and post-condition formulas over a state, and actions compose by sequence, nondeterministic choice, and bounded iteration. Partial-correctness assertions express Hoare-style contracts of structured programs, which are checked mechanically via a (bounded) weakest-liberal-precondition computation, and SAT solving. While technically neither Electrum (Alloy 6) nor DynAlloy extend the expressiveness of standard Alloy [8], both extensions improve practicality, declarativeness (the way behavioral constraints are expressed), and more importantly, the efficiency with which specifications that involve dynamic behavior can be analyzed.

3 Related Work

SysML v1 and OCL to Alloy. UML2Alloy [1] established how OCL navigation and collection operations map (approximately) onto relational logic. In [2] Alloy verifies model transformation rules. A NIST report [9] defined a methodology translating SysML v1 behavior diagrams into relational logic, using Alloy to verify system executability via behavioral abstractions. A systematic mapping of the field [24] documents a substantial body of SysML formalization work and finds that structural and behavioral analyses are almost always carried out separately rather than in a single verification run.

SysML v2 Formal Verification. Recent work has begun to address specifically the formal verification of SysML v2. Molnár et al. [17] provide an initial roadmap toward verifying SysML v2 models, identifying key challenges such as the semantics of actions, state machine analysis, and parametric constraints, and outlining

Author Proof

potential verification back-ends. However, their proposal remains at the level of a research agenda rather than a concrete translation or implemented tool chain. Other approaches for encoding SysML structural features into contract-based frameworks [7] exist, but as noted below, these target flat state-variable models and lack native support for rich structural navigation.

Dedicated Behavioral Checkers. State-machine analyzers handle behavior natively but are control-state-centric. LTSA [16] checks reachability and safety/progress via compositional reachability analysis of FSP-specified transition systems; NuSMV [6] and nuXmv [5] perform model checking of CTL/LTL over finite- and infinite-state machines. Spin [11] offers efficient LTL model checking, but its Promela language is geared toward process-algebraic descriptions rather than relational constraints. More generally, these tools offer poor support for sophisticated state definitions, such as object-oriented structures with multiplicities, part decomposition, and typed collections. None of them provides the structural and navigational vocabulary that SysML v2 attaches to its state machines—multiplicities, feature subsetting, quantified navigation over typed collections, etc. While contract-based frameworks can encode some SysML features [7], they are rooted in flat state variables, without native support for structural constraints such as reachability invariants, multiplicities, or quantification over typed collections constructs, straightforward to express relationally. Using these tools forces one to project the structure away or re-encode it by hand, in contrast to Alloy, where structural constraints are directly supported and can be naturally combined with behavioral reasoning.

Behavior and Dynamic Extensions of Alloy. Where behavioral verification exists for SysML v1 in Alloy, state machines are typically encoded with auxiliary "time" signatures (e.g., via the `ordering` module) and decoupled from the structural model. Two principled extensions add dynamics natively: Electrum / Alloy 6 [4] adds LTL over traces, and DynAlloy [10] adds an action layer with a weakest-precondition semantics. Both have been used for stateful systems, but neither has been connected to SysML v2.

4 Limitations of Direct Alloy Encodings for SysML V2

A classical application of consistency checking and instance generation is its use to identify contradictions, as well as missing constraints. As mentioned earlier, Alloy fits well for the analysis of structural aspects of a specification. Let us consider the fire-alarm model shown in Listing 1. The structural translation of this model into Alloy is the well-understood part: `Sensor`, `Controller`, and `FireAlarmSystem` become signatures, parts become relations, and multiplicities become facts, as shown in Fig. 1.a.

The model, as written, carries a defect that simple type-checking does not catch. The developer's intent is that the controller observes exactly the system's physical sensors, but there is no constraint in the model stating such relationship.

Author Proof

When using Alloy analyzer to query for the consistency of the Alloy model obtained from the SysML (Fig. 1.a) we obtain an instance where the controller within a fire alarm is wired to "loose" sensors, disjoint from the physical sensors of the system (Fig. 1.b).

The following `subsets` constraint in the fire alarm controller fixes this problem:

```
part ctrl : Controller { part sensors subsets physicalSensors; }
```

which is mapped into a corresponding constraint in Alloy:

```
fact Subsetting { all f: FireAlarmSystem | f.ctrl.sensors in f.physicalSensors }
```

Other structural features of SysML v2, such as multiple inheritance and feature redefinition, pose additional encoding challenges, but these are largely incremental over the existing SysML v1-to-Alloy experience. Parts, ports, and related notions are straightforward to map into Alloy signatures and relations. The most significant difficulties, however, arise from the behavioral aspects of SysML v2.

4.1 State-Machine Queries and Behavioral Consistency

The structural encodings discussed above address only part of what SysML v2 models capture. SysML v2 places a stronger emphasis on behavioral models, and with it come questions that structural consistency checking alone cannot answer: is every state of a state machine reachable from the initial state? Does a safety invariant hold along all executions? These are not questions that a constraint over a single configuration can answer—they require reasoning over sequences of states.

Consider the fire-alarm controller. At a minimum, consistency of the behavioral model typically requires that all states are reachable from the initial state. One may also want to check stronger properties, such as that whenever the controller is active, at least one sensor has detected a fire (`NoFalseAlarm`). These are natural properties for a behavioral model, but they are not captured by traditional structural encodings, which evaluate constraints over configurations rather than over executions.

Standard Alloy can in principle encode behavioral consistency constraints, but doing so requires explicit trace characterizations using sequences. For instance, encoding the fire-alarm controller in standard Alloy would require introducing a State signature and making the controller's location a relation indexed by State, as shown in Listing 2. This encoding uses models of sequences explicitly, and besides reducing declarativeness—which is a minor issue for a backend notation—it is generally less efficient unless manually improved. Electrum, incorporated into Alloy 6, provides a more appropriate dedicated notation for dynamic behavior, whose encoding into standard Alloy is systematized and made efficient. Studies have shown that Alloy 6 offers performance advantages over explicit sequence-based encodings in standard Alloy [8]. Encoding state machines using the new modeling elements of Alloy 6, namely mutable fields and

Author Proof

linear temporal logic (LTL) constraints, is rather direct: mutable fields represent the current control location and other mutable elements of the model (note the var keyword on currLoc and isDetecting), transition constraints are expressed as predicates, and the transition relation is asserted with LTL's "always". Reachability becomes an "eventually" property to be queried for satisfiability. For the fire-alarm controller, this encoding is concise and directly captures the properties of interest, as shown in Listing 3.

```
abstract sig Bool {}
one sig True, False extends Bool {}

sig Sensor   { detected: one Bool }
sig Controller { sensors: some Sensor }
sig FireAlarmSystem {
  physicalSensors: set Sensor,
  ctrl:          one Controller
}

fact Structure {
  all f: FireAlarmSystem | #f.physicalSensors = 3
}
```

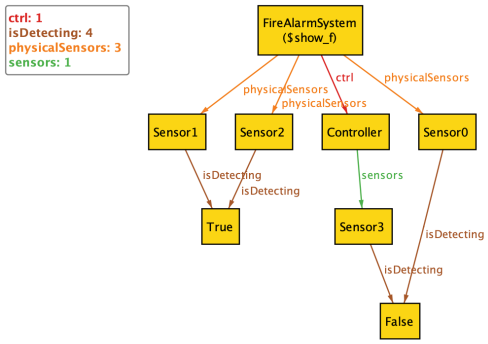


Fig. 1: (a) Structural Alloy model as directly translated from SysML. (b) An Alloy instance witnessing a missing structural constraint in the SysML model.

4.2 Navigation over Ordered and Nonunique Collections

SysML v2 expressions can navigate over collections that may be ordered and/or nonunique (sequences, ordered sets, bags), with operations such as `select`, `collect`, and `size` composed into a single expression. Formulated as a relational formula that exploits Alloy's set-based semantics and relational composition—which is often sufficient for structural constraints—the encoding can fail for these navigation-heavy expressions, because Alloy's semantics collapses duplicates, and the value it computes can differ from the one SysML v2 prescribes.

Consider a weighted majority guard: critical sensors are wired into a voting list, where a sensor may appear more than once, and the controller activates when a strict majority of the (weighted) votes report detection. In SysML v2, this is a guard on the transition from `Idle` to `Active`:

```
part def Controller {
    part sensors : Sensor[1..*];

    ref votes : Sensor[1..*] nonunique;

    accept Tick
        if (votes->select(s | s.detected)->
              size()) * 2 > votes->size()
        then active;
}
```

A relational formulation of the guard would be:

```
sig Controller { votes: set Sensor } % already collapses duplicates
all c: Controller | mul[#(c.votes & detected.True), 2] > #(c.votes)
```

increase font size, without exceeding line length

Author Proof

This encoding is incorrect. Suppose `votes = [s1, s1, s2]` with `s1` detecting and `s2` not. SysML v2 counts votes with multiplicity: two detecting votes out of three, so the controller activates. In Alloy, `votes` is a set: the duplicate `s1` cannot be represented, so `#(c.votes & detected.True)` is 1 and `#(c.votes)` is 2, giving $1 \cdot 2 \not> 2$. The encoding silently turns an activating condition into a non-activating one.

An accurate Alloy formulation would need to re-implement the multiset iteration explicitly, for example using sequences. DynAlloy offers a natural and efficient alternative: a short program walks the voting list and counts occurrences, preserving multiplicity, as shown in Listing 4. The activation decision is then taken on the exact count.

This example illustrates a broader point: navigation-heavy constraints that involve ordered or nonunique collections are not faithfully modeled by relational composition alone. DynAlloy's support for explicit iteration over collections provides a practical, efficient, and maintainable way to encode such constraints within the Alloy ecosystem.

5 Limitations and Future Work

SysML v2 poses obstacles for Alloy-based analysis that SysML v1-to-Alloy techniques do not fully cover. SysML v2 places a stronger emphasis on behavioral models, and dedicated checkers, while strong on control state, offer little support for the structure that SysML v2 fuses with behavior. We have argued that dynamic extensions of Alloy—standard Alloy for structural invariants, Alloy 6 for reachability and safety of state machines, and DynAlloy for richer control flow and navigation-heavy constraints—offer a more natural and practical foundation for analyzing SysML v2 models.

Several limitations remain. First, no single semantic framework integrates Electrum and DynAlloy. They operate under different semantics: Electrum uses mutable fields with LTL over traces, while DynAlloy is built around action composition and weakest-precondition reasoning. This means that properties that span both layers, such as a DynAlloy-verified activity establishing a condition relied upon by an Electrum state-machine invariant, cannot be checked in a unified way. Closing this gap via a common semantics or translation is an open

Listing 2: Behavioral standard Alloy model for fire alarm.

```
open util/ordering[State] as ord
enum Loc { Idle, Active, Alarm, Disabled }
sig State {
  system: one FireAlarmSystem,
  currLoc: one Loc
}
fact Init { let s = ord/first | s.currLoc =
     Idle }

pred detect[s: State] {
  let s2 = next[s] {
    let f = s.system, f2 = s2.system {
      s.currLoc = Idle and (some sens: f.ctrl
           .sensors | sens.isDetecting = True
           ) and s2.currLoc = Active and f2.
           ctrl.isOn = True
    }
  }
}
-- Similar predicates for tick, clear, stay
     ...

fact Steps {
  all s: State - last | detect[s] or tick[s]
       or clear[s] or stay[s]
}
run reach_Alarm { some s: State | s.currLoc =
      Alarm } for 4 but 4 State
```

Author Proof

challenge. Second, while DynAlloy is often significantly more efficient than Electrum for behavioral modeling—particularly when the control flow of the state machine is complex—the decision of which backend to use for a given fragment is unclear. DynAlloy's weakest-precondition computation exploits control flow to simplify SAT checks, reducing temporal search to localized contract checks. Electrum, by contrast, searches over entire traces, which can be more expensive. However, for simple reactive state machines with flat transitions and inductive properties, Electrum's encoding is straightforward and performs well. Developing heuristics or static criteria to guide this choice automatically is a topic for future investigation.

Listing 3: Behavioral Alloy 6 model for fire alarm

```
enum Loc { Idle, Active, Alarm, Disabled }

sig Sensor { var isDetecting: Bool }

sig FireAlarmSystem {
  physicalSensors: set Sensor,
  ctrl: one Controller,
  var currLoc: one Loc
}

fact Init { all f: FireAlarmSystem | f.
     currLoc = Idle }

pred detect[f: FireAlarmSystem] {
  f.currLoc = Idle and (some sens: f.ctrl.
       sensors | sens.isDetecting = True) and
       f.currLoc' = Active
}

-- Similar predicates for tick, clear, stay
     ...

fact Steps { always (all f: FireAlarmSystem |
      detect[f] or tick[f] or clear[f] or
     stay[f]) }

run reach_Alarm { eventually some f:
     FireAlarmSystem | f.currLoc = Alarm }
     for 4
```

Finally, the navigation-heavy constraints illustrate a broader issue: SysML v2's expression language includes operations over ordered and nonunique collections without direct counterparts in relational logic. While DynAlloy provides a practical way to encode these explicitly, the translation from SysML v2 expressions to DynAlloy programs is not yet systematized, and doing so for the full language remains future work.

Listing 4: Counting over the weighted voting multiset as a DynAlloy program.

```
program countDetections[c: Controller] {
  n := 0 ; i := 0 ;
  while ( i < c.votes.size ) {
    if ( c.votes[i].detected = True ) then n
         := n.plus[1] ;
    i := i.plus[1]
  }
}

assertCorrectness majorityVoting[c:
     Controller] {
  pre { true }
  countDetections[c]
  post { majorityReached iff ( n.mul[2] > c.
       votes.size ) }
}
```

Next steps are to close the Alloy 6–DynAlloy integration gap, automate the pipeline, and evaluate on larger case studies, in particular activity-rich behaviors in automated driving systems, and define a precise LTS semantics for SysML v2's behavioral subset.

Author Proof

References

1. Anastasakis, K., Bordbar, B., Georg, G., Ray, I.: UML2Alloy: a challenging model transformation. In: Model Driven Engineering Languages and Systems: 10th International Conference, MoDELS 2007. Lecture Notes in Computer Science, vol. 4735, pp. 436–450. Springer (2007) Küster, J.
2. Anastasakis, K., Bordbar, B., ~~Georg, G., Ray, I.~~: Analysis of model transformations via Alloy. In: Models in Software Engineering: Workshops and Symposia at MoDELS 2009, pp. 116–130. Springer (2010)
3. Barbau, R., Bock, C.E.: Verifying executability of SysML behavior models using satisfiability modulo theory solvers. Tech. Rep. NIST IR 8283, National Institute of Standards and Technology, Gaithersburg, MD (2020). https://doi.org/10.6028/NIST.IR.8283
4. Brunel, J., Chemouil, D., Cunha, A., Macedo, N.: The Electrum analyzer: model checking relational first-order temporal specifications. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE 2018), pp. 884–887. ACM (2018)
5. Cavada, R., et al.: The nuxmv symbolic model checker. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. Lecture Notes in Computer Science, vol. 8559, pp. 334–342. Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_22
6. Cimatti, A., et al.: NuSMV 2: an opensource tool for symbolic model checking. In: Brinksma, E., Larsen, K.G. (eds.) CAV 2002. LNCS, vol. 2404, pp. 359–364. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45657-0_29
7. Cimatti, A., Dorigatti, M., Tonetta, S.: OCRA: a tool for checking the refinement of temporal contracts. In: Denney, E., Bultan, T., Zeller, A. (eds.) 2013 28th IEEE/ACM International Conference on Automated Software Engineering, ASE 2013, Silicon Valley, CA, USA, November 11-15, 2013, pp. 702–705. IEEE (2013). https://doi.org/10.1109/ASE.2013.6693137
8. Cornejo, C., Regis, G., Aguirre, N., Frias, M.F.: A study of the electrum and DynAlloy dynamic behavior notations. IEEE Trans. Softw. Eng. **49**(11), 4946–4963 (2023)
9. Doerr, J., Bock, C., Barbau, R.: Verifying executability of SysML behavior models using Alloy analyzer. NIST Interagency/Internal Report (NISTIR) 8388-upd1, National Institute of Standards and Technology (2024). https://doi.org/10.6028/NIST.IR.8388-upd1
10. ~~Frias, M.F., López Pombo, C.G., Aguirre, N.M., Maibaum, T.S.E.~~: DynAlloy: upgrading alloy with actions. In: Proceedings of the 27th International Conference on Software Engineering (ICSE 2005), pp. 442–451. ACM (2005) (a)
11. Holzmann, G.J.: The model checker SPIN. IEEE Trans. Softw. Eng. **23**(5), 279–295 (1997)
12. Jackson, D.: Software Abstractions: Logic, Language, and Analysis, MIT Press (~~2012~~). ~~2nd edn~~ revised edition 2016
13. Jackson, D.: Alloy: a language and tool for exploring software designs. Commun. ACM **62**(9), 66–76 (2019)
14. Jackson, D., Jha, S., Damon, C.: Isomorph-free model enumeration: a new method for checking relational specifications. ACM Trans. Program. Lang. Syst. **20**(2), 302–343 (1998)

(a) Frias, M.F., Galeotti, J.P., López Pombo, C., Aguirre, N.

Author Proof

15. Jackson, D., Shlyakhter, I., Sridharan, M.: A micromodularity mechanism. In: Tjoa, A.M., Gruhn, V. (eds.) Proceedings of the 8th European Software Engineering Conference held jointly with 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering 2001, Vienna, Austria, September 10-14, 2001, pp. 62–73. ACM (2001). https://doi.org/10.1145/503209.503219
16. Magee, J., Kramer, J.: Concurrency: State Models and Java Programs, Wiley (2006). 2nd edn
17. Molnár, V., et al.: Towards the formal verification of sysml v2 models. In: Wimmer, M., Egyed, A., Combemale, B., Chechik, M. (eds.) Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems, MODELS Companion 2024, Linz, Austria, September 22-27, 2024, pp. 1086–1095. ACM (2024). https://doi.org/10.1145/3652620.3687820
18. Object Management Group: Kernel Modeling Language (KerML) (~~2023~~ 2025). https://www.omg.org/spec/KerML/
19. Object Management Group: OMG Systems Modeling Language (SysML), Version 2 (~~2023~~ 2025). https://www.omg.org/spec/SysML/
20. Object Management Group (OMG): OMG Systems Modeling Language (OMG SysML™), Version 1.0 (2007). http://www.omg.org/spec/SysML/1.0/
21. Regis, G., et al.: DynAlloy analyzer: a tool for the specification and analysis of Alloy models with dynamic behaviour. In: Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017), pp. 969–973. ACM, Paderborn, Germany (2017)
22. Schäfer, M., Gogolla, M.: Enhancing development and consistency of UML models and model executions with USE studio. In: Guerra, E., Iovino, L. (eds.) MODELS '20: ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems, Virtual Event, Canada, 18-23 October, 2020, Companion Proceedings, pp. 14:1–14:5. ACM (2020). https://doi.org/10.1145/3417990.3422011
23. Torlak, E., Jackson, D.: Kodkod: a relational model finder. In: Grumberg, O., Huth, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 13th International Conference, TACAS 2007, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2007 Braga, Portugal, March 24 - April 1, 2007, Proceedings. LNCS, vol. 4424, pp. 632–647. Springer (2007). https://doi.org/10.1007/978-3-540-71209-1_49
24. Wolny, S., Mazak, A., Carpella, C., Geist, V., Wimmer, M.: Thirteen years of SysML: a systematic mapping study. Softw. Syst. Model. **19**(1), 111–169 (2020). https://doi.org/10.1007/s10270-019-00735-y
25. Yeung, W.L.: Checking consistency between UML class and state models based on CSP and B. J. Univers. Comput. Sci. **10**(11), 1540–1559 (2004)

Author Proof

Author Queries

Chapter 8

Query Refs.	Details Required	Author's response
AQ1	This is to inform you that corresponding author and email address have been identified as per the information available in the Copyright form.	OK
AQ2	As Per Springer style, both city and country names must be present in the affiliations. Accordingly, we have inserted the city names in all affiliations. Please check and confirm if the inserted city names are correct. If not, please provide us with the correct city names.	Please see corrections on front page